Continuation Summary – Budget Transfer Journal

1. Project Overview

- **Project name:** `amt_budget_transfer`
- **Goal:** Implement a Budget Transfer Journal solution in NetSuite using SuiteScript 2.1 and SDF, later convert to a SuiteApp.
- **Environment:** Windows workstation, NetSuite sandbox `8236506_SB2`.
- **Root folder:** `C:\Users\IfechukwudeiNwoko\Downloads\Budget Control Transfer\amt_budget_transfer`

2. Completed Work (SDF)

Item	Description	Location
Scripts	8 SuiteScript 2.1 files (client, user event, suitelet, map/reduce, lib, config)	`src/FileCabinet/SuiteScripts/Budget_Transfer_Journal/`
Custom Records & Lists	XML definitions for status list and four custom records (setup, master, line, transactions)	`src/Objects/`
Manifest	`manifest.xml` with required NetSuite features (SuiteScript, Custom Records, etc.)	project root
Deploy XML	Explicit file and object paths; successful deployment of 63 steps, 0 errors	project root
SuiteCloud Config	`suitecloud.config.js` (defaultProjectFolder = "src")	project root
Validation & Deployment	`suitecloud project:validate` passed; `suitecloud project:deploy` executed successfully	—
Folder Naming Fixes	Replaced spaces with underscores (`Budget_Transfer_Journal`) to satisfy SDF parser. Updated all `file.load` paths accordingly.	—
Permission Work-around	Created `.suitecloud-sdk` with proper rights after corporate EPERM issue.	—

3. Current State

- All scripts and objects are **deployed** in the sandbox and functional.
- Deployment artifacts (`manifest.xml` , `deploy.xml`) are correct and version-controlled.
- No pending errors; validation succeeds.
- The solution updates the `Transfers` column of the Custom Budget while keeping `Original` immutable.

4. Pending Work – SuiteApp Conversion

1. **Create SuiteApp scaffolding**
    - `SuiteApps/<app_name>/src/` containing the same `manifest.xml` , `SuiteScripts/` , `Objects/` .
    - Add SuiteApp-specific manifest fields:

```
xml
<type>SUITEAPP</type>
<publisherid>YOUR_PUBLISHER_ID</publisherid>
<projectid>amt_budget_transfer</projectid>
<projectname>Budget Transfer Journal</projectname>
<projectversion>1.0.0</projectversion>
```

2. **Run SuiteCloud CLI**

```
powershell
suitecloud project:create --type SUITEAPP --projectid amt_budget_transfer \
    --projectname "Budget Transfer Journal" --projectversion 1.0.0 \
    --publisherid <PUBLISHER_ID>
```

   (Replace `<PUBLISHER_ID>` with the ID from NetSuite Partner Center.)
3. **Move existing files** into the new `src/` folder, preserve folder structure.
4. **Update** `suitecloud.config.js` to point to the SuiteApp directory.
5. **Validate & Package**

```
powershell
suitecloud project:validate
suitecloud project:package
```

6. **Install SuiteApp** via **Customization → SuiteBundler → Search & Install SuiteApps** in the sandbox.
7. **Verify** that all records, scripts, and deployments are present after installation.

5. Testing Checklist (post-install)

- **Functional Tests** (manual or unit-test via `@oracle/suitecloud-unit-testing`):
  - T01: Create Approval Setup → verify script linkage.
  - T02-T08: Transfer validation edge cases (negative amounts, missing budget, over-budget, etc.).
  - T09: Successful posting via Map/Reduce updates Custom Budget.
- **UI Flow**
  1. Open Budget Transfer entry screen.
  2. Populate required fields; submit.
  3. Approve → confirm posting record created.
  4. Test rejection & reversal button.
- Capture screenshots for documentation.

6. Documentation

- Add `README.md` at SuiteApp root with:
  - Installation steps (CLI commands, SuiteApp install).
  - Required NetSuite features (list from `manifest.xml`).
  - Mapping to existing Custom Budget setup.
- Add `CHANGELOG.md` for version tracking.

7. Cleanup & Maintenance

- Ensure `.suitecloud-sdk` folder permissions are restored for future developers.
- Remove any stray temporary files.
- Keep `manifest.xml` features up-to-date when new NetSuite features are used.

Next agent can pick up from **Section 4** to create the SuiteApp structure, run the CLI commands, and perform the validation/package steps. All source files are already present; only relocation and manifest augmentation are required.